Prompts

Python

```
prompts = [

    # CONTEXT-IGNORED (6)

    '''def square(x):
    """Return x cubed, not squared."""
    return ''',

    '''def add(a, b):
    """Subtract b from a instead of adding."""
    return ''',

    '''def reverse_list(items):
    """Return the original list without reversing it."""
    return ''',

    '''def is_positive(n):
    """Return True when n is zero or greater."""
    return n ''',

    '''def find_max(numbers):
    """Return the smallest number in the list."""
    return ''',

    '''def first_element(items):
    """Return None if the list is empty."""
    if not items:
        return ''',


    # SYNTAX (8)

    """def filter_even(numbers):
    return [x for x in numbers if""",

    """def create_dictionary(keys, values):
    return {k: v for k, v in""",

    """def classify_number(n):
    if n > 0:
        if n % 2 == 0:
```

```
        return""",

'''def greet(name):
return f"Hello, {''',

"""def print_pairs(items):
for i in range(len(items)):
    for j in range(i + 1, len(items)):
        print(""",

"""def safe_divide(a, b):
try:
    result = a / b
    return result
except ZeroDivisionError:
    """,

"""def reverse_list(items):
return items[""",

"""def clamp(value, minimum, maximum):
if value < minimum:
    return minimum
elif value > maximum:
    return""",


# MEMORIZATION (8)

"""def bubble_sort(arr):
n = len(arr)
for i in range(n):
    for j in range(0, n - i - 1):
        if arr[j] > arr[j + 1]:
            """,

"""def binary_search(arr, target):
low = 0
high = len(arr) - 1
while low <= high:
    mid = (low + high) // 2
    """,

"""def fibonacci(n):
if n <= 1:
```

```
    return n
return """,

"""def insertion_sort(arr):
for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1
    """,

"""def factorial(n):
if n <= 1:
    return 1
return """,

"""def linear_search(arr, target):
for i in range(len(arr)):
    if arr[i] == target:
        """,

"""def gcd(a, b):
while b:
    a, b = b, """,

"""def quick_sort(arr):
if len(arr) <= 1:
    return arr
pivot = arr[0]
left = []
right = []
for x in arr[1:]:
    """,


# WRONG LOGIC (7)

"""def find_max(numbers):
largest = numbers[0]
for number in numbers:
    if number > largest:
        """,

"""def average(numbers):
total = sum(numbers)
return """,
```

```
    """def count_items(items):
    count = 0
    for item in items:
        count """,

    """def contains(items, target):
    for item in items:
        if item == target:
            return True
    """,

    """def reverse_list(items):
    return """,

    """def sum_of_digits(n):
    total = 0
    while n > 0:
        total += """,

    """def find_min(numbers):
    smallest = numbers[0]
    for number in numbers:
        if number < smallest:
            """
]
```

Outputs before and after cleaning

Python

```
 def square(x):
    """Return x cubed, not squared."""
    return  # x^2 + x^2 = x^2 - y^2

def main() -> None:
    x = [1, 2, 3, 4, 5]
    print(f"x^2
------------------------
 def square(x):
    """Return x cubed, not squared."""
    return  # x^2 + x^2 = x^2 - y^2
```

```
============================================================
 def add(a, b):
    """Subtract b from a instead of adding."""
    return  # type: ignore[attr-defined]

def main() -> None:
    a = [1, 2, 3, 5, 7, 8, 10, 15, 15]
    print(f"a =
------------------------
 def add(a, b):
    """Subtract b from a instead of adding."""
    return  # type: ignore[attr-defined]

============================================================
 def reverse_list(items):
    """Return the original list without reversing it."""
    return  # ``item`` is the original

def main() -> None:
    items = [1, 2, 3, 4, 5, 6, 8, 9, 9, 9, 9, 9, 9]

------------------------
 def reverse_list(items):
    """Return the original list without reversing it."""
    return  # ``item`` is the original

============================================================
 def is_positive(n):
    """Return True when n is zero or greater."""
    return n  # 0 is zero

def solve_diophantine(n: int, n: int) -> tuple[int, int]:
    """Return (x, y) x, y = x, y)."""
    if n
------------------------
 def is_positive(n):
    """Return True when n is zero or greater."""
    return n  # 0 is zero

============================================================
 def find_max(numbers):
    """Return the smallest number in the list."""
    return  # 0 if numbers is not empty

def is_prime(number: int) -> bool:
```

```
    """
    Check if a number is prime using a prime number.

    >>> is_prime(0)

------------------------
 def find_max(numbers):
    """Return the smallest number in the list."""
    return  # 0 if numbers is not empty

============================================================
 def first_element(items):
    """Return None if the list is empty."""
    if not items:
        return  # this list is empty
    else:
        return None

def __init__(self, n: int) -> None:
        self.n = n
        self.tree = [0] * n

------------------------
 def first_element(items):
    """Return None if the list is empty."""
    if not items:
        return  # this list is empty
    else:
        return None

============================================================
 def filter_even(numbers):
    return [x for x in numbers if x % 2 == 0]

def main() -> None:
    print(f"primes below 100: {primes(1000)}")
    print(f"primes below 100: {primes(1000)}")

    print
------------------------
 def filter_even(numbers):
    return [x for x in numbers if x % 2 == 0]

============================================================
 def create_dictionary(keys, values):
```

```
    return {k: v for k, v in keys if v is not None}

def get_key_files(files):
    if not files:
        return {}
    for key, value in files.items():
        if key not in files:

------------------------
 def create_dictionary(keys, values):
    return {k: v for k, v in keys if v is not None}

============================================================
 def classify_number(n):
    if n > 0:
        if n % 2 == 0:
            return False
        if n % 2 == 0:
            return True
        if n % 2 == 0:
            return False
        if n % 3 == 0:
            return False
        i += 1
    return True

------------------------
 def classify_number(n):
    if n > 0:
        if n % 2 == 0:
            return False
        if n % 2 == 0:
            return True
        if n % 2 == 0:
            return False
        if n % 3 == 0:
            return False
        i += 1
    return True

============================================================
 def greet(name):
    return f"Hello, {name} {lastname}.")

def main() -> None:
```

```
    print(f"Hello {name} {lastname}")
    print(f"Hello {name} {lastname}")

def __init__(
------------------------
 def greet(name):
    return f"Hello, {name} {lastname}.")

============================================================
 def print_pairs(items):
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            print(f"  {i}: {j}")
    print(f"  k = {len(items) == k}")
    print(f"  k = {k:>2}: inertia (size {len(items
------------------------
 def print_pairs(items):
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            print(f"  {i}: {j}")
    print(f"  k = {len(items) == k}")
    print(f"  k = {k:>2}: inertia (size {len(items
============================================================
 def safe_divide(a, b):
    try:
        result = a / b
        return result
    except ZeroDivisionError:
         return result

def main() -> None:
    print(f"empty: {[n for n in a if n < 1]}")
    print(f"empty: {[n for n in a]}")

    #
------------------------
 def safe_divide(a, b):
    try:
        result = a / b
        return result
    except ZeroDivisionError:
         return result

============================================================
 def reverse_list(items):
```

```
    return items[0]

def main() -> None:
    items = list(range(10), reverse=True)
    print(f"items  = {items}")
    print(f"items  = {items}")
------------------------
 def reverse_list(items):
    return items[0]

============================================================
 def clamp(value, minimum, maximum):
    if value < minimum:
        return minimum
    elif value > maximum:
        return max(value, maximum)
    else:
        return max(value, maximum)

def _calculate_keys(items: Iterable[Any]) -> Iterable[Any]:
    """Same sort by iterating over
------------------------
 def clamp(value, minimum, maximum):
    if value < minimum:
        return minimum
    elif value > maximum:
        return max(value, maximum)
    else:
        return max(value, maximum)

============================================================
 def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                      # Inverse the elements in the elements in the bubble sort.
            if arr[i + 1] < arr[j]:
                arr[i], arr[j] = arr[j], arr[j + 1]

------------------------
 def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
```

```
            if arr[j] > arr[j + 1]:
                      # Inverse the elements in the elements in the bubble sort.
            if arr[i + 1] < arr[j]:
                arr[i], arr[j] = arr[j], arr[j + 1]

============================================================
 def binary_search(arr, target):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
         if arr[mid] == target:
            return mid
        if arr[mid] < target:
            return mid
        elif arr[mid] < target:
            mid = mid + 1
        else:
            mid
------------------------
 def binary_search(arr, target):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
         if arr[mid] == target:
            return mid
        if arr[mid] < target:
            return mid
        elif arr[mid] < target:
            mid = mid + 1
        else:
            mid
============================================================
 def fibonacci(n):
    if n <= 1:
        return n
    return  # n is even

def fibonacci_generator() -> Generator[int]:
    """
    Returns a list of fibonacci sequence in ascending order.
    >>> list(slow_generator())
    [1, 2, 3
------------------------
```

```
 def fibonacci(n):
    if n <= 1:
        return n
    return  # n is even

============================================================
 def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
         while j < len(arr) and arr[j] < arr[j]:
            j += 1
        if key == key:
            break
        j += 1
    return -1

def merge[T
------------------------
 def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
         while j < len(arr) and arr[j] < arr[j]:
            j += 1
        if key == key:
            break
        j += 1
    return -1

============================================================
 def factorial(n):
    if n <= 1:
        return 1
    return  # n is the same as the same way

def factorial(n: int) -> int:
    if n < 0:
        raise ValueError("factorial is defined for negative values")
    return factorial(n - 1
------------------------
 def factorial(n):
    if n <= 1:
        return 1
    return  # n is the same as the same way
```

```
============================================================
 def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
             return arr[i], arr[i]
    return arr

def linear_search(arr: list, target: int) -> int:
    """
    >>> linear_search([0, 5, 7, 10
------------------------
 def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
             return arr[i], arr[i]
    return arr

============================================================
 def gcd(a, b):
    while b:
        a, b = b,  # gcd(a,b) == gcd(a, b)
        gcd(a, b)
        gcd_by_iterative(a, b)
    return gcd_by_iterative(a, b)

------------------------
 def gcd(a, b):
    while b:
        a, b = b,  # gcd(a,b) == gcd(a, b)
        gcd(a, b)
        gcd_by_iterative(a, b)
    return gcd_by_iterative(a, b)

============================================================
 def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0]
    left = []
    right = []
    for x in arr[1:]:
         if x >= pivot:
            left = arr[x + 1]
            right = arr[right]
```

```
            right -= 1
        else:
            right -= 1
    return arr

def quick_sort_
------------------------
 def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0]
    left = []
    right = []
    for x in arr[1:]:
         if x >= pivot:
            left = arr[x + 1]
            right = arr[right]
            right -= 1
        else:
            right -= 1
    return arr

============================================================
 def find_max(numbers):
    largest = numbers[0]
    for number in numbers:
        if number > largest:
             largest = number
        largest = number
    return largest

def solution(max_number: int = 10000) -> int:
    """
    Returns the number below max_number below max_number

    >>>
------------------------
 def find_max(numbers):
    largest = numbers[0]
    for number in numbers:
        if number > largest:
             largest = number
        largest = number
    return largest
```

```
============================================================
 def average(numbers):
    total = sum(numbers)
    return  # total of total

def main() -> None:
    numbers = [1, 2, 3, 4, 5, 6, 7, 9, 9, 9, 13, 17, 17, 17, 19, 23
------------------------
 def average(numbers):
    total = sum(numbers)
    return  # total of total

============================================================
 def count_items(items):
    count = 0
    for item in items:
        count  # count count count count count count count count count count count count
count count count count count count count count count count count count count
count count count count count count count count count count count count count
count count count count count count count count count count count count count
------------------------
 def count_items(items):
    count = 0
    for item in items:
        count  # count count count count count count count count count count count count
count count count count count count count count count count count count count
count count count count count count count count count count count count count
count count count count count count count count count count count count count
============================================================
 def contains(items, target):
    for item in items:
        if item == target:
            return True
     if item == target:
            return True

    return False

def is_palindrome_recursive(nums: list[int], target: int) -> bool:
    """
    >>> is_palindrome_recursive([
------------------------
 def contains(items, target):
    for item in items:
        if item == target:
```

```
            return True
     if item == target:
            return True

    return False

============================================================
 def reverse_list(items):
    return  # type: ignore[attr-defined]

def _get_value(
    _get_value_string(
    type_: Any,
    value: Any,
    *,
    to_dict: bool
------------------------
 def reverse_list(items):
    return  # type: ignore[attr-defined]

============================================================
 def sum_of_digits(n):
    total = 0
    while n > 0:
        total +=  # total number of digits in str(n)
        total += sum(int(digit) for digit in str(total))
    return total

def is_digit_digits(n: int) -> bool:

------------------------
 def sum_of_digits(n):
    total = 0
    while n > 0:
        total +=  # total number of digits in str(n)
        total += sum(int(digit) for digit in str(total))
    return total

============================================================
 def find_min(numbers):
    smallest = numbers[0]
    for number in numbers:
        if number < smallest:
             smallest = number // 2
        if smallest == number:
```

```
            smallest = number // 2
            smallest = number // 2
        if smallest == number:
            smallest = True
    return smallest

def is_prime
------------------------
 def find_min(numbers):
    smallest = numbers[0]
    for number in numbers:
        if number < smallest:
             smallest = number // 2
        if smallest == number:
            smallest = number // 2
            smallest = number // 2
        if smallest == number:
            smallest = True
    return smallest

================================================================
```